AI FIRST NATIONS / MANGROVE AI

Mangrove Species Classification HD Model Evaluation Specification

Independent Jupyter Notebook assessment | 100-mark rubric | Application-readiness verification

Specification	Requirement
Model	EfficientNet-B0 (3-class image classifier)
Primary output	Reproducible .ipynb notebook with quantitative evidence
Decision	Evidence-based deployment recommendation: Ready / Experimental / Not Ready
Assessment status	Proposed HD-standard rubric; not an official RMIT marking scheme

1. Purpose and evaluation questions

Develop a research-quality evaluation of the Mangrove Species Classification model already used in the Mangrove AI pipeline. The aim is to measure genuine predictive performance on independent drone imagery and establish whether the saved weights and preprocessing operate correctly in Flask. High reported accuracy alone does not establish field readiness.

- How accurately can the model distinguish Orange, Red and Yellow categories on unseen drone imagery?
- Are all three categories represented fairly, especially the least-represented class?
- Does the model learn canopy/species characteristics rather than site, lighting, survey or background cues?
- Are tiled-image aggregation and the Flask inference outputs reliable and reproducible?

2. Required developer deliverables

Deliverable	Required evidence
Notebook	Mangrove_Species_Evaluation.ipynb; executed outputs and narrative interpretation
Data audit	Manifest with original image IDs, site/survey identifiers, labels, tile lineage and splits
Metrics	Machine-readable CSV/JSON metrics, confusion matrix, per-image predictions
Visual evidence	Learning curves, calibration, PR/ROC where valid, failure cases, tiled-image examples
Application tests	Actual saved model loaded and compared with Flask inference
Handover	Requirements/config, seed, model checksum, reproducible run instructions

3. Mandatory notebook sections

01. Research design

Define scope, labels, intended use, hypotheses, evaluation unit (tile vs source image) and acceptance criteria.

02. Dataset audit

Inventory original images, tiles, site/survey IDs, annotation provenance, label balance, resolution, quality and duplicates.

03. Split integrity

Group by source image; where feasible hold out entire survey/site. Demonstrate no source/tile overlap across partitions.

04. Model reproducibility

Record architecture, classifier head, weight hash, class-index mapping, transforms, batch size, device, seeds and package versions.

05. Core predictive metrics

Calculate all mandatory classification metrics on a locked independent test set; explain denominators and aggregation.

06. Tile-to-image analysis

Evaluate per-tile and full-source-image predictions separately; test aggregation and edge/coverage behaviour.

07. Robustness and subgroup analysis

Report performance by site, canopy, acquisition conditions and difficult cases when labels/sample sizes permit.

08. Controlled improvements

Compare original baseline against justified changes using validation data only; do not tune on test data.

09. Flask end-to-end verification

Test actual model loading, tiler, output schema, runtime, memory, failure states and prediction consistency.

10. Conclusions and readiness

Summarise evidence, uncertainty, error analysis, known limits, acceptance checks and final recommendation.

4. HD marking rubric (100 marks)

Criterion	Marks	HD standard
Research questions and evaluation design	5	Specific, measurable, deployment-linked questions and valid methodology
Dataset audit and annotation quality	10	Comprehensive provenance, distributions, duplicate/label review
Leakage prevention and splits	15	Documented group-wise splits; site holdout where possible; verified isolation
Reproducibility and model setup	10	Exact saved weights, transforms, class order, seeds and environment
Classification metrics and calibration	20	Correct class-wise metrics, confusion matrix, confidence calibration and interpretation
Robustness, confounding and failure analysis	15	Realistic negatives/shift tests, error taxonomy and subgroup limitations
Controlled improvement experiments	10	Fair baseline comparison; validation-only tuning and reasoned conclusions
Flask integration and operational tests	10	Verified production-path predictions, latency, resource and error tests
Notebook clarity and conclusions	5	Well-presented, executable, critical and evidence-based

Grading descriptors: HD 80-100 = rigorous and reproducible with independent testing and critical analysis; D 70-79 = comprehensive with minor gaps; C 60-69 = adequate metrics but limited robustness; P 50-59 = basic evaluation; below 50 = unreliable or materially incomplete.

5. Mandatory quantitative metrics

Metric	Mandatory interpretation
3x3 confusion matrix	Rows actual / columns predicted, with fixed Orange/Red/Yellow order
Precision / Recall / F1 per class	Report support and identify weakest class
Macro F1 / balanced accuracy	Primary imbalance-aware indicators
Overall accuracy / weighted F1	Secondary measures; cannot replace per-class analysis
One-vs-rest ROC-AUC / PR-AUC	Where valid and supported; avoid invalid computation for absent classes
Log loss / Brier score / ECE	Evaluate probability reliability and overconfidence
Top-class confidence distributions	Correct vs incorrect predictions, especially high-confidence errors
Full-image species accuracy	Separately from tile accuracy and only on eligible mangrove images
Latency and peak memory	Measure on actual application hardware

Formulas: Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2PR/(P+R); Accuracy = correct predictions / total predictions. For three classes, compute metrics per class and report macro averages. Undefined metrics must be reported as undefined, not silently replaced with zero.

6. Evaluation experiments - mandatory

A. Reproduce the inherited baseline

Load best_mangrove_model.pth and fixed class mapping. Independently reproduce inference; compare with earlier reported approximately 97% test accuracy without assuming equivalence of datasets.

B. Per-class and imbalance audit

Examine Orange, Red and Yellow sample counts, misclassifications and class-wise confidence; emphasise minority-class recall.

C. Source/site confounding

Test whether image site, survey, lighting, camera or background correlates with species labels. Hold out sources/sites where data permits.

D. Robustness across imagery

Test shadows, dense canopy, crown size, lighting, blur and image quality; separate valid species-labelled imagery from out-of-scope scenes.

E. Tile aggregation and mixed scenes

Compare tile predictions vs averaged image-level probabilities; inspect images with mixed canopy/categories. Do not claim multi-species detection from one image-level label.

F. Controlled improvement

Compare two justified changes (class balancing, transforms, augmentation, calibration, aggregation, or architecture) using validation only.

7. Data protocol and leakage controls

- Create a manifest: source_image_id, tile_id, site_id, survey_id, acquisition metadata if permitted, label, split and annotation provenance.
- Split at source-image level before tiling; preferably test cross-site or cross-survey generalisation. Do not split sibling tiles across train and test.
- Review exact and near-duplicate imagery, overlapping orthomosaic areas, and temporally adjacent frames that may leak scene identity.
- Freeze a held-out test set and preserve an experiment log. Threshold tuning, calibration and model selection must use validation data only.
- Report test sample sizes and class support at original-image level, not merely the number of highly correlated tiles.
- For confidence intervals, bootstrap by independent source image or site/survey (not tile) where sample size permits.
- Respect Indigenous data governance: keep survey data within approved storage and do not upload to unapproved annotation/cloud services.
- Check class-index consistency against the model checkpoint and source dataset. Never assume alphabetical fallback labels are necessarily the correct mapping.
- Verify whether Orange/Red/Yellow are formal species names or project-specific categories before using biological claims.

- Audit training code for shared transform state between train and validation subsets; explicitly verify augmentations and evaluation transforms.

8. Performance targets and acceptance gates

The following are proposed engineering targets, not promises or universal scientific thresholds. Confirm them with the project team before running the locked final test. A model may receive an HD evaluation even when it fails these targets, provided the methodology and conclusions are rigorous.

Measure	Proposed target
Macro F1 (source-image level)	>= 0.90
Per-class recall	>= 85% for every supported class
Balanced accuracy	>= 0.90
Held-out site evaluation	Completed where feasible
Calibration	Reliability plot and score reported
Flask critical integration checks	All pass

A deployment recommendation must also consider independent sample size, false-result consequences, site coverage and runtime. The model is not production-validated solely by reaching the numerical targets.

9. Required plots and examples

- Source-level train/validation/test distributions and class counts
- Representative original images and their tile lineage
- Confusion matrix (raw counts and normalised)
- Class-wise precision/recall/F1 with support
- Confidence calibration / reliability diagram
- ROC/PR curves where statistically valid
- Correct high-confidence and incorrect high-confidence examples
- Per-site or per-condition results with sample sizes
- Baseline vs improved-model comparison
- Full-image versus tile-level prediction comparison
- Flask output screenshot paired with notebook output and numerical checks

Every important figure must have a short interpretation: what it shows, why it matters, and what uncertainty or limitation remains.

10. Flask application verification

Test the actual application inference path, not just a standalone model call: Upload -> image validation -> shared image_tiler.py -> preprocessing -> saved EfficientNet-B0 weights -> tile aggregation -> response object -> results page. The species classifier should run only when the upstream binary result permits it under the application policy.

- Exact model checkpoint and class ordering are identical between notebook and Flask.
- Input transforms (resize to 224x224, normalisation, channel handling) and tile settings are consistent.
- Predicted probabilities and image-level labels match within a documented floating-point tolerance.

- Large, small, edge-sized, mixed-content and corrupted images are handled predictably.
- Run cold-start and warm-inference latency plus peak RAM; state hardware, batch size and image dimensions.
- When a model is unavailable, the app shows a safe unavailable state rather than invented output.
- For species: verify upstream non-mangrove rejection and unsupported/mixed-category limitations.
- Provide a pass/fail table with test ID, input, expected, observed, evidence and issue status.

11. Final results summary - required template

Metric	Baseline	Final	Target / criterion	Pass?
Orange F1	TBD	TBD	Pre-agreed	TBD
Red F1	TBD	TBD	Pre-agreed	TBD
Yellow F1	TBD	TBD	Pre-agreed	TBD
Macro F1	TBD	TBD	Pre-agreed	TBD
Balanced accuracy	TBD	TBD	Pre-agreed	TBD
Overall accuracy	TBD	TBD	Pre-agreed	TBD
Per-class recall floor	TBD	TBD	Pre-agreed	TBD
Log loss / ECE	TBD	TBD	Pre-agreed	TBD
Held-out site performance	TBD	TBD	Pre-agreed	TBD
Flask integration	TBD	TBD	Pre-agreed	TBD

12. Non-negotiable HD integrity requirements

- No leakage between train and test at source-image level.
- No accuracy claim without independent labelled ground truth and denominators.
- No threshold/model tuning using final test data.
- No reporting only overall accuracy while hiding minority-class errors.
- No unqualified claims about species identification, health, carbon or field reliability.
- No claims of application readiness without running the actual Flask integration checks.
- No invented metrics, fabricated screenshots, cherry-picked results or omitted failure cases.
- Notebook must execute top-to-bottom with clear paths, dependency versions and reproducible outputs.

13. Final developer sign-off checklist

[ ] Independent source-level test set verified

[ ] Annotation/label mapping audited

[ ] All mandatory metrics and calibration completed

[ ] Failure cases and realistic field conditions analysed

[ ] Controlled experiments compared fairly

[ ] Full-image and tile-level predictions differentiated

[ ] Flask inference and runtime tested

[ ] Outputs saved as figures + CSV/JSON

[ ] Notebook reproducible in clean environment

[ ] Deployment recommendation justified

14. Final assessment statement

Conclude with: (1) measured independent test performance; (2) performance variation across conditions/sites; (3) main failure modes and risks; (4) agreement between notebook and deployed Flask inference; (5) a justified status of Ready for defined use, Experimental/Conditional, or Not Ready. Distinguish model confidence from verified real-world accuracy.